

TECHNICAL TRAINING COURSE

JAXB — Java API for XML Binding

Java 11 · Tomcat 9.0 · JAXB 2.3 (javax.xml.bind)



Topics Covered: Data Binding · XML Schema · Two Paths · Compilation
Type Mapping · Annotations · Marshal/Unmarshal · Object Models

Platform: Java 11 LTS | Tomcat 9.0 | JAXB 2.3.x

Namespace: javax.xml.bind.* (NOT jakarta.xml.bind.*)

Binding files: <http://java.sun.com/xml/ns/jaxb> version="2.1"

Duration: 1 Day (8 hours)

Level: Intermediate – Java Developer

Table of Contents

Table of Contents	2
1. The Need for Data Binding	4
1.1 Learning Objectives	4
1.2 The Problem with Manual XML Processing	4
1.3 What is Data Binding?	4
1.4 Benefits of Data Binding	5
1.5 JAXB in the Java Ecosystem	5
1.6 Review Questions	5
2. XML Schema	7
2.1 Learning Objectives	7
2.2 What is XML Schema?	7
2.3 XSD Document Structure	7
2.4 XSD Built-in Simple Types	7
2.5 Cardinality and Occurrence Constraints	8
2.6 Review Questions	8
3. Two Paths	9
3.1 Learning Objectives	9
3.2 Overview of the Two Paths	9
3.3 Path 1 — Schema-First (XSD → Java)	9
3.4 Path 2 — Java-First (Annotations)	10
3.5 Choosing the Right Path	10
3.6 Review Questions	10
4. JAXB Compilation	11
4.1 Learning Objectives	11
4.2 The xjc Compiler	11
4.3 Generated Artifacts	11
4.4 Binding Customisation Files (.xjb)	12
4.5 Maven Integration (Java 11 / Tomcat 9)	13
4.6 Review Questions	13
5. Mapping Schema Types to Java	15
5.1 Learning Objectives	15
5.2 SimpleType Mappings	15
5.3 Java Time Types (Java 11 / JAXB 2.3)	15
Option A — Binding Customisation	15
Option B — XmlAdapter (Recommended for Java 11)	16
5.4 ComplexType Mappings	16
5.5 Inheritance — xs:extension	16

5.6 Review Questions	17
6. Java to XML Mapping Using Annotations	18
6.1 Learning Objectives	18
6.2 Complete Annotation Reference.....	18
6.3 @XmlAccessorType — Default Access Strategies	18
6.4 Controlling Collections with @XmlElementWrapper.....	19
6.5 Package-Level Namespace Declaration.....	19
6.6 Custom Type Adapters	19
6.7 Review Questions	20
7. Marshalling and Unmarshalling	21
7.1 Learning Objectives	21
7.2 JAXBContext	21
7.3 Marshalling — Java to XML.....	21
7.4 Unmarshalling — XML to Java	22
7.5 Schema Validation During Unmarshalling	22
7.6 Review Questions	23
8. Working with JAXB Object Models.....	24
8.1 Learning Objectives	24
8.2 Building Object Graphs with ObjectFactory	24
8.3 JAXB in a Tomcat 9 Servlet.....	24
8.4 JAXB with StAX — Partial Document Processing	25
8.5 Performance Best Practices	25
8.6 Review Questions	26
Appendix A — Quick Reference.....	27
JAXB 2.3 Annotation Cheat Sheet.....	27
Maven Dependency Block — Java 11 / JAXB 2.3.....	27

1. The Need for Data Binding

Package Namespace — All code uses javax.xml.bind.* (NOT jakarta.xml.bind.*)

This document targets Java 11 / Tomcat 9, which require the javax.xml.bind.* namespace.

```
import javax.xml.bind.JAXBContext; import javax.xml.bind.Marshaller;
import javax.xml.bind.annotation.*; import javax.xml.bind.annotation.adapters.*;
import javax.servlet.*; import javax.servlet.annotation.WebListener;
```

Binding files use: xmlns:jaxb="http://java.sun.com/xml/ns/jaxb" version="2.1"

Do NOT use jakarta.xml.bind.* or https://jakarta.ee/xml/ns/jaxb — those are for Tomcat 10+.

1.1 Learning Objectives

- Understand why manual XML parsing does not scale in enterprise applications
- Articulate the definition of data binding and its key benefits
- Distinguish between data binding libraries and their Java EE equivalents

1.2 The Problem with Manual XML Processing

In Java, XML documents can be processed using low-level APIs such as DOM, SAX, and StAX. While these APIs give complete control over the XML structure, they impose a heavy burden on developers when the goal is simply to move data between a Java object model and an XML document.

Consider a simple Order object with ten fields. Using DOM to convert it to XML requires creating a DocumentBuilder and an empty Document, creating each element and attribute individually, appending every node to its correct parent, and writing the document to an OutputStream with a Transformer. The reverse process requires the same amount of code again, including null-checks for every optional field. The resulting code is verbose, fragile, and tightly coupled to the schema.

The Maintenance Problem

When the schema changes (e.g., an element is renamed or a new child added), every piece of parsing code that touches that element must be located and updated manually. In a large codebase this becomes a significant source of defects.

1.3 What is Data Binding?

Data binding is the process of automatically mapping between a structured data format (XML, JSON) and an in-memory object model (Java classes). A data binding framework generates or interprets mapping metadata and performs the conversion, removing the need for hand-written parsing code.

Term	Definition
Marshalling	Converting a Java object graph into an XML document. Also called serialisation.
Unmarshalling	Converting an XML document into a Java object graph. Also called deserialisation.
Schema	A formal description of the legal structure of an XML document (XSD).

Binding

The correspondence between an XML Schema type and a Java class or property.

1.4 Benefits of Data Binding

- Dramatic reduction in boilerplate — one marshal/unmarshal call replaces dozens of DOM statements
- Type safety — JAXB maps XSD types to Java types; type errors surface at compile time
- Schema validation — the framework can validate XML against the schema during unmarshalling
- Separation of concerns — business logic operates on Java objects; binding layer handles XML
- Maintainability — regenerate binding classes when the schema changes; no manual parsing updates
- Tool support — IDEs understand generated classes; debuggers can inspect object state

1.5 JAXB in the Java Ecosystem

Version	Specification	Namespace	Status with Java 11
JAXB 2.0	JSR 222	javax.xml.bind.*	Removed from JDK 11+
JAXB 2.2	JSR 222	javax.xml.bind.*	Removed from JDK 11+
JAXB 2.3	JSR 222	javax.xml.bind.*	Add as Maven dep — use this one
JAXB 3.x	Jakarta	jakarta.xml.bind.*	Tomcat 10 / Jakarta EE 9+ only

Java 11 / Tomcat 9 — Required Maven Dependency

JAXB was removed from the JDK in Java 9 (JEP 320). On Java 11 with Tomcat 9 you must add the JAXB API and a runtime implementation explicitly. Use the javax.xml.bind groupId — NOT jakarta.xml.bind:

```
<!-- pom.xml -->
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>
<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
  <version>2.3.9</version>
  <scope>runtime</scope>
</dependency>
```

1.6 Review Questions

1. Describe two specific problems with using DOM to serialise a 20-field Java object to XML.
2. Why was JAXB removed from the JDK in Java 9, and what must Java 11 projects do to use it?
3. Define marshalling and unmarshalling in your own words.

2. XML Schema

2.1 Learning Objectives

- Understand the purpose and structure of XML Schema Definition (XSD) documents
- Write XSD schemas that define complex types, simple types, and cardinality constraints
- Apply common XSD restrictions (patterns, enumerations, value ranges)
- Understand how schema structure drives JAXB class generation

2.2 What is XML Schema?

XML Schema Definition (XSD) is a W3C recommendation that defines the vocabulary and structure of an XML document. A schema answers: which elements and attributes are permitted; what data type each holds; which are required vs optional; how many times a repeating element can appear; and what constraints apply to values.

2.3 XSD Document Structure

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:tns="http://example.com/orders"
  targetNamespace="http://example.com/orders"
  elementFormDefault="qualified">
  <!-- Simple type with restriction -->
  <xs:simpleType name="OrderIdType">
    <xs:restriction base="xs:string">
      <xs:pattern value="ORD-[0-9]{4}-[0-9]{3}" />
    </xs:restriction>
  </xs:simpleType>
  <!-- Enumeration -->
  <xs:simpleType name="StatusType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="PENDING" />
      <xs:enumeration value="CONFIRMED" />
      <xs:enumeration value="SHIPPED" />
      <xs:enumeration value="CANCELLED" />
    </xs:restriction>
  </xs:simpleType>
  <!-- Complex type -->
  <xs:complexType name="AddressType">
    <xs:sequence>
      <xs:element name="street" type="xs:string" />
      <xs:element name="city" type="xs:string" />
      <xs:element name="postCode" type="xs:string" />
      <xs:element name="country" type="xs:string" default="GB" />
    </xs:sequence>
  </xs:complexType>
  <xs:element name="order" type="tns:OrderType" />
</xs:schema>
```

2.4 XSD Built-in Simple Types

XSD Type	Java Type	Notes
xs:string	String	UTF-8 string
xs:int	int / Integer	32-bit signed integer
xs:long	long / Long	64-bit signed integer
xs:decimal	BigDecimal	Exact decimal; use for money
xs:double	double / Double	IEEE 754 floating point
xs:boolean	boolean / Boolean	true / false
xs:date	XMLGregorianCalendar	ISO 8601 date: yyyy-MM-dd
xs:dateTime	XMLGregorianCalendar	ISO 8601 datetime
xs:base64Binary	byte[]	Base64-encoded binary data
xs:anyURI	String	URI / URL value
xs:positiveInteger	BigInteger	Integer > 0

2.5 Cardinality and Occurrence Constraints

Term	Definition
minOccurs="0" maxOccurs="1"	Optional element (0 or 1 occurrences). Java field is nullable.
minOccurs="1" maxOccurs="1"	Required element (exactly 1). Default if both omitted.
minOccurs="0" maxOccurs="unbounded"	Zero or more. JAXB generates List<T>.
minOccurs="1" maxOccurs="unbounded"	One or more. JAXB generates List<T>.

2.6 Review Questions

4. What XSD element is used to declare a type that restricts xs:string to a set of allowed values?
5. What Java type does JAXB map xs:decimal to, and why is this preferable to xs:double for monetary values?
6. Write a minimal XSD complexType for a 'Product' with a required name (string), optional description (string), and price (decimal).

3. Two Paths

3.1 Learning Objectives

- Understand and compare the schema-first and Java-first (annotation-driven) development paths
- Choose the appropriate path for a given project scenario
- Understand the tools associated with each path

3.2 Overview of the Two Paths

Aspect	Schema-First (XSD → Java)	Java-First (Java → XML)
Starting Point	XSD document(s)	Annotated Java classes
Primary Tool	xjc (JAXB compiler)	JAXB annotations + runtime
Contract Owner	Schema team / standards body	Java developer
Interoperability	Highest — schema is authoritative	Good if annotations used carefully
Best For	B2B, standards, SOAP services	Internal tools, rapid prototyping

3.3 Path 1 — Schema-First (XSD → Java)

In the schema-first path, the XSD is the authoritative contract. Java classes are generated from it using the xjc compiler. Generated classes should not be hand-edited, as they will be overwritten on the next xjc run.

Maven Plugin — jaxb2-maven-plugin version 2.5.0

Use version 2.5.0 of jaxb2-maven-plugin — NOT version 3.x.

Version 3.x targets Jakarta EE 10 and generates jakarta.xml.bind.* imports, which are wrong for Java 11 / Tomcat 9.

Version 2.5.0 generates the correct javax.xml.bind.* imports.

Also note: use <schemaDirectory> and <schemaFiles>, not <sources><source>.

Use <bindingIncludes><bindingInclude>NONE to prevent auto-discovery of .xjb files.

```
<plugin>
<groupId>org.codehaus.mojo</groupId>
<artifactId>jaxb2-maven-plugin</artifactId>
<version>2.5.0</version>  <!-- NOT 3.x -->
<executions>
  <execution>
    <goals><goal>xjc</goal></goals>
  </execution>
</executions>
<configuration>
  <schemaDirectory>src/main/xsd</schemaDirectory>
  <schemaFiles>orders.xsd</schemaFiles>
  <!-- Prevent auto-processing of .xjb files -->
  <bindingIncludes>
    <bindingInclude>NONE</bindingInclude>
  </bindingIncludes>
</configuration>
</plugin>
```

```
</bindingIncludes>
<packageName>com.example.model</packageName>
<outputDirectory>
  ${project.build.directory}/generated-sources/jaxb
</outputDirectory>
<clearOutputDir>false</clearOutputDir>
</configuration>
</plugin>
```

3.4 Path 2 — Java-First (Annotations)

In the Java-first path, a developer writes (or already has) Java classes and adds JAXB annotations to control how they are mapped to XML. This path is fastest to get started but risks schema inconsistency over time.

3.5 Choosing the Right Path

Term	Definition
Use Schema-First when...	...the schema is published externally (WSDL, industry standard, B2B partner contract).
Use Schema-First when...	...multiple consumers (Java, .NET, Python) must interoperate against a fixed contract.
Use Java-First when...	...you own both producer and consumer and want rapid iteration.
Use Java-First when...	...you are adding XML output to an existing Java domain model.

3.6 Review Questions

7. In a project where an external trading partner provides the schema, which path should you use and why?
8. What tool generates Java from XSD? What tool generates XSD from Java?
9. Name two risks of the Java-first approach.

4. JAXB Compilation

4.1 Learning Objectives

- Run xjc from the command line to compile an XSD into Java classes
- Configure xjc using binding customisation files
- Understand the artifacts xjc generates and the role of each
- Integrate xjc into a Maven build using the jaxb2-maven-plugin

4.2 The xjc Compiler

xjc (XML-to-Java Compiler) is the reference implementation of the JAXB binding compiler. On Java 11 it must be run from the JAXB standalone distribution because it was removed from the JDK.

```
# Basic: compile a single schema
xjc -p com.example.model \
    -d src/main/java \
    src/main/xsd/orders.xsd

# With a binding customisation file
xjc -p com.example.model \
    -b bindings.xjb \
    -d src/main/java \
    src/main/xsd/orders.xsd

# Useful flags:
# -p <pkg>      Java package for generated classes
# -d <dir>      Output directory
# -b <file>     External binding customisation file
# -verbose      Print progress information
```

4.3 Generated Artifacts

For a typical orders.xsd, xjc generates the following files. Note: generated classes import from javax.xml.bind.annotation.* (NOT jakarta.xml.bind.annotation.*):

Term	Definition
Order.java	JAXB-annotated class mapping to OrderType. Contains all fields, getters, setters.
Address.java	Class for AddressType.
StatusType.java	Java enum for the StatusType simpleType.
ObjectFactory.java	Factory class with create*() methods for every generated type.
package-info.java	Package-level @XmlSchema annotation declaring the namespace.

Example of a generated class — note the correct javax.xml.bind.annotation.* import:

```
// Generated by xjc - do not edit manually
package com.example.model;
```

```

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import javax.xml.datatype.XMLGregorianCalendar;
import javax.xml.bind.annotation.*; // javax — NOT jakarta

@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(name = "OrderType", propOrder = {
    "orderId", "status", "orderDate",
    "deliveryAddress", "lineItem", "notes"
})
public class Order {
    @XmlElement(required = true)
    protected String orderId;
    @XmlElement(required = true)
    @XmlSchemaType(name = "string")
    protected StatusType status;
    @XmlElement(required = true)
    @XmlSchemaType(name = "date")
    protected XMLGregorianCalendar orderDate;
    @XmlElement(required = true)
    protected Address deliveryAddress;
    @XmlElement(required = true)
    protected List<LineItem> lineItem;
    protected String notes;
    @XmlAttribute(name = "currency", required = true)
    protected String currency;
    public List<LineItem> getLineItem() {
        if (lineItem == null) lineItem = new ArrayList<>();
        return lineItem;
    }
}

```

4.4 Binding Customisation Files (.xjb)

Binding customisations allow you to override xjc defaults. Binding files must use the JAXB 2.x namespace — NOT the Jakarta EE 3.x namespace:

Binding file namespace — important

Correct (JAXB 2.x / Java 11 / Tomcat 9):

```
xmlns:jaxb="http://java.sun.com/xml/ns/jaxb" version="2.1"
```

Wrong (Jakarta EE 3.x / Tomcat 10+ only):

```
xmlns:jaxb="https://jakarta.ee/xml/ns/jaxb" version="3.0"
```

The jaxb2-maven-plugin 2.5.0 expects the 2.1 namespace.
Using the 3.0 namespace will cause the plugin to fail.

```

<!-- bindings.xjb - JAXB 2.x namespace -->
<jaxb:bindings
  xmlns:jaxb="http://java.sun.com/xml/ns/jaxb"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  version="2.1">

  <jaxb:bindings schemaLocation="orders.xsd" node="/xs:schema">
    <!-- Rename a generated class -->
    <jaxb:bindings node="//xs:complexType[@name='OrderType']">
      <jaxb:class name="PurchaseOrder"/>
    </jaxb:bindings>
  </jaxb:bindings>

```

```

    </jaxb:bindings>
    <!-- Rename a generated property -->
    <jaxb:bindings node="//xs:element[@name='lineItem']">
      <jaxb:property name="lineItems"/>
    </jaxb:bindings>
  </jaxb:bindings>
</jaxb:bindings>

```

4.5 Maven Integration (Java 11 / Tomcat 9)

Complete Maven POM configuration for JAXB 2.3 with Java 11. Use javax.xml.bind groupId — NOT jakarta.xml.bind:

```

<properties>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
</properties>
<dependencies>
  <!-- JAXB API — javax.xml.bind namespace -->
  <dependency>
    <groupId>javax.xml.bind</groupId>
    <artifactId>jaxb-api</artifactId>
    <version>2.3.1</version>
  </dependency>
  <!-- JAXB Runtime -->
  <dependency>
    <groupId>com.sun.xml.bind</groupId>
    <artifactId>jaxb-impl</artifactId>
    <version>2.3.9</version>
    <scope>runtime</scope>
  </dependency>
</dependencies>
<build><plugins>
  <plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>jaxb2-maven-plugin</artifactId>
    <version>2.5.0</version>    <!-- NOT 3.x -->
    <executions>
      <execution><goals><goal>xjc</goal></goals></execution>
    </executions>
    <configuration>
      <schemaDirectory>src/main/xsd</schemaDirectory>
      <schemaFiles>orders.xsd</schemaFiles>
      <bindingIncludes>
        <bindingInclude>NONE</bindingInclude>
      </bindingIncludes>
      <packageName>com.example.model</packageName>
      <outputDirectory>
        ${project.build.directory}/generated-sources/jaxb
      </outputDirectory>
    </configuration>
  </plugin>
</plugins></build>

```

4.6 Review Questions

10. What xjc flag sets the Java package for generated classes?
11. Name four classes generated by xjc for the orders.xsd example and describe each.

12. What is the purpose of ObjectFactory.java?
13. Why must you use jaxb2-maven-plugin version 2.5.0 rather than 3.x for Java 11 / Tomcat 9?

5. Mapping Schema Types to Java

5.1 Learning Objectives

- Understand how xjc maps XSD simple types, complex types, and derivation hierarchies to Java
- Handle XSD date/time types and the XMLGregorianCalendar workaround for Java 11
- Understand how enumerations, unions, and lists are handled

5.2 SimpleType Mappings

XSD Simple Type	Required Field	Optional Field	Notes
xs:string	String	String	Always a reference type; null for absent optional
xs:int	int	Integer	Unboxed when required; boxed when optional
xs:long	long	Long	Same boxing rule
xs:boolean	boolean	Boolean	
xs:decimal	BigDecimal	BigDecimal	Always BigDecimal — exact arithmetic
xs:date	XMLGregorianCalendar	XMLGregorianCalendar	See java.time note below
xs:base64Binary	byte[]	byte[]	JAXB handles encode/decode automatically
xs:positiveInteger	BigInteger	BigInteger	

5.3 Java Time Types (Java 11 / JAXB 2.3)

A significant pain point with JAXB 2.3 on Java 11 is that XSD date/time types map to XMLGregorianCalendar by default. Two approaches to use java.time types instead:

Option A — Binding Customisation

Map xs:date to java.time.LocalDate using a binding file. Remember to use the JAXB 2.x namespace in the binding file:

```
<!-- bindings.xjb - JAXB 2.x namespace (NOT jakarta.ee) -->
<jaxb:bindings
  xmlns:jaxb="http://java.sun.com/xml/ns/jaxb"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  version="2.1"
  schemaLocation="orders.xsd">
  <jaxb:bindings node="//xs:element[@name='orderDate']">
    <jaxb:property>
      <jaxb:baseType name="java.time.LocalDate"/>
    </jaxb:property>
  </jaxb:bindings>
</jaxb:bindings>
```

```
</jaxb:bindings>
</jaxb:bindings>
```

Option B — XmlAdapter (Recommended for Java 11)

An `XmlAdapter` bridges `XMLGregorianCalendar` and `java.time` types at runtime. Import from `javax.xml.bind.annotation.adapters` (NOT `jakarta.*`):

```
package com.example.adapter;

import javax.xml.bind.annotation.adapters.XmlAdapter; // javax - NOT jakarta
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class LocalDateAdapter extends XmlAdapter<String, LocalDate> {
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ISO_LOCAL_DATE; // yyyy-MM-dd

    @Override
    public LocalDate unmarshal(String v) {
        return v == null ? null : LocalDate.parse(v, FMT);
    }

    @Override
    public String marshal(LocalDate v) {
        return v == null ? null : v.format(FMT);
    }
}

// Usage in the domain class:
@XmlElement
@XmlJavaTypeAdapter(LocalDateAdapter.class)
private LocalDate orderDate;
```

5.4 ComplexType Mappings

Term	Definition
xs:sequence	Ordered sequence of child elements. Maps to a class with fields in <code>propOrder</code> .
xs:all	Any-order group of elements. Maps to a class with no <code>propOrder</code> constraint.
xs:choice	Exactly one of a set of elements.
xs:complexContent	Extension or restriction of another <code>complexType</code> — maps to Java inheritance.

5.5 Inheritance — xs:extension

```
<!-- XSD: ShippedOrderType extends OrderType -->
<xs:complexType name="ShippedOrderType">
  <xs:complexContent>
    <xs:extension base="tns:OrderType">
      <xs:sequence>
```

```
        <xs:element name="trackingNumber" type="xs:string"/>
        <xs:element name="shippedDate" type="xs:date"/>
    </xs:sequence>
</xs:extension>
</xs:complexContent>
</xs:complexType>

// Generated Java (note: javax.xml.bind.annotation.* import):
@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(name = "ShippedOrderType", propOrder = {
    "trackingNumber", "shippedDate"
})
public class ShippedOrder extends Order {
    @XmlElement(required = true)
    protected String trackingNumber;
    @XmlElement(required = true)
    @XmlSchemaType(name = "date")
    protected javax.xml.datatype.XMLGregorianCalendar shippedDate;
}
```

5.6 Review Questions

14. What Java type does xjc use for xs:decimal? Why is this important for financial applications?
15. Why is XMLGregorianCalendar problematic for new Java 11 code, and what is the recommended workaround?
16. How does xjc represent an XSD complexType that extends another complexType in Java?

6. Java to XML Mapping Using Annotations

6.1 Learning Objectives

- Apply the full set of JAXB 2.3 annotations to control XML mapping
- Understand `@XmlAccessorType` and its effect on default property discovery
- Implement `XmlAdapter` for custom type conversions
- Control XML namespace mapping at class and package level

6.2 Complete Annotation Reference

All annotations are imported from `javax.xml.bind.annotation.*` (NOT `jakarta.xml.bind.annotation.*`):

Term	Definition
<code>@XmlRootElement</code>	Marks a class as a top-level XML element. Required for marshalling without <code>ObjectFactory</code> .
<code>@XmlType</code>	Maps a class to an XSD <code>complexType</code> . Controls <code>propOrder</code> , <code>name</code> , <code>namespace</code> .
<code>@XmlElement</code>	Maps a field/property to an XML child element.
<code>@XmlAttribute</code>	Maps a field/property to an XML attribute.
<code>@XmlValue</code>	Maps a field to the text content of an element.
<code>@XmlTransient</code>	Excludes a field from all XML mapping.
<code>@XmlAccessorType</code>	Sets the default access strategy: <code>FIELD</code> , <code>PROPERTY</code> , <code>PUBLIC_MEMBER</code> , or <code>NONE</code> .
<code>@XmlEnum</code>	Marks an enum for XSD enumeration mapping.
<code>@XmlElementWrapper</code>	Wraps a collection in a parent element.
<code>@XmlJavaTypeAdapter</code>	Specifies an <code>XmlAdapter</code> subclass for custom type conversion.
<code>@XmlSchema</code>	Package-level annotation setting the namespace and <code>elementFormDefault</code> .

6.3 @XmlAccessorType — Default Access Strategies

```
// FIELD access - annotate fields directly
// All imports from javax.xml.bind.annotation.*
import javax.xml.bind.annotation.*;

@XmlRootElement(name = "product")
@XmlType(propOrder = { "sku", "name", "price", "category" })
@XmlAccessorType(XmlAccessType.FIELD)
public class Product {
    @XmlAttribute(name = "sku", required = true)
    private String sku;
    @XmlElement(required = true)
    private String name;
    @XmlElement(required = true)
    private java.math.BigDecimal price;
```

```

@XmlElement(name = "cat")
private String category;
@XmlTransient
private String internalCode;
// getters/setters
}

```

6.4 Controlling Collections with @XmlElementWrapper

```

@XmlRootElement(name = "catalogue")
@XmlAccessorType(XmlAccessType.FIELD)
public class Catalogue {
    // WITH wrapper - produces:
    // <catalogue>
    //   <products>
    //     <product>...</product>
    //   </products>
    // </catalogue>
    @XmlElementWrapper(name = "products")
    @XmlElement(name = "product")
    private List<Product> products;
}

```

6.5 Package-Level Namespace Declaration

To avoid repeating namespace strings on every annotation, declare the namespace at the package level in package-info.java. Import from javax.xml.bind.annotation.* (NOT jakarta.*):

```

// package-info.java
@XmlSchema(
    namespace = "http://example.com/catalogue",
    elementFormDefault = XmlNsForm.QUALIFIED,
    xmlns = {
        @XmlNs(prefix = "cat",
            namespaceURI = "http://example.com/catalogue")
    }
)
package com.example.model;

import javax.xml.bind.annotation.XmlNs; // javax - NOT jakarta
import javax.xml.bind.annotation.XmlNsForm;
import javax.xml.bind.annotation.XmlSchema;

```

6.6 Custom Type Adapters

```

// MoneyAdapter - marshals BigDecimal as a formatted currency string
import javax.xml.bind.annotation.adapters.XmlAdapter;

public class MoneyAdapter extends XmlAdapter<String, java.math.BigDecimal> {
    @Override
    public java.math.BigDecimal unmarshal(String v) {
        if (v == null || v.isBlank()) return null;
        return new java.math.BigDecimal(v)
            .setScale(2, java.math.RoundingMode.HALF_UP);
    }
    @Override

```

```
public String marshal(java.math.BigDecimal v) {  
    if (v == null) return null;  
    return v.setScale(2, java.math.RoundingMode.HALF_UP).toPlainString();  
}  
  
// In the domain class:  
@XmlElement(required = true)  
@XmlJavaTypeAdapter(MoneyAdapter.class)  
private java.math.BigDecimal unitPrice;
```

6.7 Review Questions

17. What is the effect of `@XmlAccessorType(XmlAccessType.NONE)`, and when would you use it?
18. Write the annotations to map a `List<String>` to: `<tags><tag>value</tag></tags>`
19. How do you declare a namespace for all classes in a package without repeating it on every annotation?

7. Marshalling and Unmarshalling

7.1 Learning Objectives

- Create and configure JAXBContext, Marshaller, and Unmarshaller
- Marshal Java objects to String, File, OutputStream, and DOM
- Unmarshal XML from String, File, InputStream, URL, and DOM
- Configure schema validation during unmarshalling

7.2 JAXBContext

JAXBContext is the entry point to the JAXB API. It is thread-safe and expensive to create — create it once and reuse it. Marshaller and Unmarshaller instances are NOT thread-safe and must be created per-operation. Import from javax.xml.bind (NOT jakarta.xml.bind):

```
import javax.xml.bind.JAXBContext; // javax - NOT jakarta
import javax.xml.bind.JAXBException;

// Option 1: from a list of classes (Java-first path)
JAXBContext ctx = JAXBContext.newInstance(
    Order.class, Address.class, LineItem.class);

// Option 2: from a package name (schema-first; ObjectFactory must exist)
JAXBContext ctx = JAXBContext.newInstance("com.example.model");

// Thread-safe singleton pattern
public class JaxbHelper {
    private static final JAXBContext CTX;
    static {
        try {
            CTX = JAXBContext.newInstance(Order.class);
        } catch (JAXBException e) {
            throw new ExceptionInInitializerError(e);
        }
    }
    public static JAXBContext get() { return CTX; }
}
```

7.3 Marshalling — Java to XML

```
import javax.xml.bind.*; // javax - NOT jakarta
import java.io.*;
import java.nio.file.*;

JAXBContext ctx = JaxbHelper.get();
Marshaller marshaller = ctx.createMarshaller();
marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
marshaller.setProperty(Marshaller.JAXB_ENCODING, "UTF-8");

Order order = buildOrder();

// To a String
StringWriter sw = new StringWriter();
marshaller.marshal(order, sw);
String xml = sw.toString();
```

```
// To a File
marshaller.marshal(order, Path.of("order.xml").toFile());

// To an OutputStream (e.g., HTTP response)
try (OutputStream os = response.getOutputStream()) {
    marshaller.marshal(order, os);
}

// Wrapping a non-@XmlRootElement class with JAXBElement
ObjectFactory factory = new ObjectFactory();
JAXBElement<Order> je = factory.createOrder(order);
marshaller.marshal(je, sw);
```

7.4 Unmarshalling — XML to Java

```
import javax.xml.bind.*; // javax - NOT jakarta

JAXBContext ctx = JaxbHelper.get();
Unmarshaller unmarshaller = ctx.createUnmarshaller();

// From a File
Order order = (Order) unmarshaller.unmarshal(Path.of("order.xml").toFile());

// From a String
Order order = (Order) unmarshaller.unmarshal(
    new java.io.StringReader(xml));

// From an InputStream (e.g., HTTP request body)
try (InputStream is = request.getInputStream()) {
    Order order = (Order) unmarshaller.unmarshal(is);
}

// Type-safe with JAXBElement
JAXBElement<Order> je = unmarshaller.unmarshal(
    new javax.xml.transform.stream.StreamSource(
        new java.io.StringReader(xml)), Order.class);
Order order = je.getValue();
```

7.5 Schema Validation During Unmarshalling

```
import javax.xml.XMLConstants;
import javax.xml.validation.*;
import javax.xml.bind.ValidationEvent;

SchemaFactory sf = SchemaFactory.newInstance(
    XMLConstants.W3C_XML_SCHEMA_NS_URI);
Schema schema = sf.newSchema(
    getClass().getResource("/xsd/orders.xsd"));

Unmarshaller u = ctx.createUnmarshaller();
u.setSchema(schema);
u.setEventHandler(event -> {
    if (event.getSeverity() == ValidationEvent.FATAL_ERROR
        || event.getSeverity() == ValidationEvent.ERROR) {
        throw new RuntimeException(
            "XML validation failed: " + event.getMessage());
    }
});
```

```
    }  
    return true; // continue on WARNING  
});
```

7.6 Review Questions

20. Why should JAXBContext be created once and reused, while Marshaller and Unmarshaller are created per-operation?
21. Write the code to marshal an Order object to a String with formatted output and UTF-8 encoding.
22. How do you enable XSD validation during unmarshalling? Where should the schema file be placed in a Maven project?

8. Working with JAXB Object Models

8.1 Learning Objectives

- Build and manipulate JAXB object graphs using ObjectFactory and constructors
- Integrate JAXB with Tomcat 9 Servlet endpoints
- Use JAXB in conjunction with StAX for partial document processing
- Apply best practices for thread safety, performance, and testability

8.2 Building Object Graphs with ObjectFactory

```
// Java 11 – building an Order object graph
ObjectFactory factory = new ObjectFactory();

Order order = factory.createOrder();
order.setOrderId("ORD-2024-001");
order.setStatus(StatusType.PENDING);
order.setCurrency("GBP");

Address addr = factory.createAddress();
addr.setStreet("10 Downing Street");
addr.setCity("London");
addr.setPostCode("SW1A 2AA");
addr.setCountry("GB");
order.setDeliveryAddress(addr);

LineItem item = factory.createLineItem();
item.setSku("WIDGET-001");
item.setQty(java.math.BigInteger.valueOf(5));
item.setUnitPrice(new java.math.BigDecimal("9.99"));
order.getLineItem().add(item);
```

8.3 JAXB in a Tomcat 9 Servlet

In a Tomcat 9 Servlet environment (Java 11), JAXB is not provided by the container — you must bundle it in the WAR. All imports use javax.* (NOT jakarta.*):

```
// JaxbContextListener.java – initialise JAXBContext at startup
import javax.servlet.ServletContextEvent;           // javax – NOT jakarta
import javax.servlet.ServletContextListener;
import javax.servlet.annotation.WebListener;
import javax.xml.bind.JAXBContext;                 // javax – NOT jakarta

@WebListener
public class JaxbContextListener implements ServletContextListener {
    private static final String ATTR = "jaxbContext";
    @Override
    public void contextInitialized(ServletContextEvent sce) {
        try {
            JAXBContext ctx = JAXBContext.newInstance(Order.class);
            sce.getServletContext().setAttribute(ATTR, ctx);
        } catch (Exception e) {
            throw new RuntimeException("JAXB init failed", e);
        }
    }
}
```

```

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        sce.getServletContext().removeAttribute(ATTR);
    }
}

// OrderServlet.java
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import javax.xml.bind.*;

@WebServlet("/orders/*")
public class OrderServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws java.io.IOException {
        JAXBContext ctx = (JAXBContext)
            getServletContext().getAttribute("jaxbContext");
        try {
            Unmarshaller u = ctx.createUnmarshaller();
            Order order = (Order) u.unmarshal(req.getInputStream());
            // process order...
            resp.setStatus(HttpServletResponse.SC_CREATED);
        } catch (JAXBException e) {
            resp.sendError(HttpServletResponse.SC_BAD_REQUEST,
                e.getMessage());
        }
    }
}

```

8.4 JAXB with StAX — Partial Document Processing

```

import javax.xml.stream.*;
import javax.xml.bind.*;           // javax - NOT jakarta

XMLInputFactory xif = XMLInputFactory.newInstance();
// Security: disable external entity processing (XXE prevention)
xif.setProperty(XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false);
xif.setProperty(XMLInputFactory.SUPPORT_DTD, false);

JAXBContext ctx = JaxbHelper.get();
Unmarshaller u = ctx.createUnmarshaller();

try (InputStream is = Files.newInputStream(Path.of("orders-batch.xml"))) {
    XMLStreamReader xsr = xif.createXMLStreamReader(is);
    while (xsr.hasNext()) {
        xsr.next();
        if (xsr.isStartElement()
            && "order".equals(xsr.getLocalName())) {
            JAXBElement<Order> je = u.unmarshal(xsr, Order.class);
            processOrder(je.getValue());
            // xsr is now positioned after </order>
        }
    }
    xsr.close();
}

```

8.5 Performance Best Practices

Term	Definition
Singleton JAXBContext	Create once per class-set; store as a static final or application-scoped bean.
Per-request Marshaller/Unmarshaller	Create fresh for each operation — they are not thread-safe.
Schema caching	Compile Schema once from SchemaFactory and reuse; do not re-read the XSD per request.
Avoid JAXB_FORMATTED_OUTPUT in production	Pretty-printing adds significant whitespace overhead.
StAX for large documents	Use the StAX + unmarshal(reader, Class) pattern for documents >1 MB.

8.6 Review Questions

23. Why is the `ObjectFactory.createOrder()` approach preferable to calling `new Order()` directly for xjc-generated classes?
24. Explain why `JAXBContext` should not be created inside a Servlet's `doGet` or `doPost` method.
25. Describe the StAX + JAXB pattern for processing a batch file containing 10,000 order elements. Why is this better than a single `unmarshal()` call?

Appendix A — Quick Reference

JAXB 2.3 Annotation Cheat Sheet

Annotation	Target	Key Attributes
<code>@XmlRootElement</code>	Class	name, namespace
<code>@XmlType</code>	Class	name, namespace, propOrder
<code>@XmlAccessorType</code>	Class	value: FIELD PROPERTY PUBLIC_MEMBER NONE
<code>@XmlElement</code>	Field/Method	name, required, nillable, defaultValue
<code>@XmlAttribute</code>	Field/Method	name, required, namespace
<code>@XmlValue</code>	Field/Method	(no attributes) — maps to element text content
<code>@XmlTransient</code>	Field/Method	(no attributes) — excludes from mapping
<code>@XmlElementWrapper</code>	Field/Method	name, namespace — wraps a collection
<code>@XmlJavaTypeAdapter</code>	Field/Class	value: XmlAdapter subclass
<code>@XmlEnum</code>	Enum class	(no attributes)
<code>@XmlSeeAlso</code>	Class	value: array of subclasses
<code>@XmlSchema</code>	Package	namespace, elementFormDefault, xmlns

Maven Dependency Block — Java 11 / JAXB 2.3

Use the javax.xml.bind groupId throughout — NOT jakarta.xml.bind:

```

<!-- JAXB API - javax.xml.bind namespace (NOT jakarta.xml.bind) -->
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>
<!-- JAXB Runtime -->
<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
  <version>2.3.9</version>
  <scope>runtime</scope>
</dependency>
<!-- XJC compiler (schema-first path) -->
<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-xjc</artifactId>
  <version>2.3.9</version>
  <scope>provided</scope>
</dependency>

<!-- Maven plugin: use 2.5.0 - NOT 3.x (3.x generates jakarta.* imports) -->
<!-- Plugin configuration: use <schemaDirectory> and <schemaFiles> -->
>
<!-- Binding files: xmlns:jaxb="http://java.sun.com/xml/ns/jaxb"
version="2.1" -->

```

